#8 Check if the search region intersects current child. Again, the region's class will have to implement this check.
#9 If there is any intersection (so *there might be* points in common), we should call this method recursively on current child, and then add all the results, if any, to the list of point we will return.
#10 At this point, we can just return all the points we collected in this call.

Listing 10.20 shows a generic implementation of this method for Ss-trees, that assumes the region passed as argument includes a method to check if the region itself intersects a hyper-sphere (a node's bounding envelope). Please refer to section 9.3.6 for a detailed explanation and examples about search on the most common types of regions, and their algebraic meaning.

10.4.3 Approximated Similarity Search

As mentioned a few times now, similarity search in k-d trees, but also R-trees and Ss-trees, suffer of what is called the *curse of dimensionality*: the methods on these data structures become exponentially slower with the growth of the number of dimensions of the search space; k-d trees also suffer from additional sparsity issues that become more relevant in higher dimensions.

While using R-trees and Ss-trees can improve the balance of the trees and result in better trees and faster construction, there is still something more we can do to improve the performance of the similarity search methods.